

Strategic Decision Framework

Evaluate between ANSI SQL and SPARK-SQL for providing Analytical Workload

I would start by not treating this as a pure feature comparison between ANSI SQL and Spark SQL. For a financial services company, the first question is:

Which platform creates the least business risk while still meeting the analytical workload?

That changes the order of evaluation.

1. Start with business-critical use cases, not technology

Build the framework around 5 to 10 real workloads, for example:

- regulatory and audit reporting
 - P&L and exposure calculations
 - fraud or AML monitoring
 - customer risk scoring
 - intraday liquidity or trade surveillance
 - historical research and large backtesting jobs
- Then classify each workload by:
- data volume
 - latency requirement
 - correctness sensitivity
 - auditability requirement
 - complexity of window functions
 - frequency of reruns or restatements

This matters because ANSI-style SQL engines and Spark SQL often look similar on paper, but window-heavy workloads can behave very differently in practice.

2. Use a three-layer decision model I would evaluate in this order:

A. Technical fit

Ask: Can the platform do the work correctly and predictably?

For window analytical functions, test things like:

- ROW_NUMBER, RANK, DENSE_RANK
- LAG, LEAD
- running totals
- moving averages
- cumulative exposure
- frame clauses: ROWS BETWEEN, RANGE BETWEEN
- partitioning by account, desk, portfolio, instrument, or customer
- ordering by timestamp, trade date, settlement date, sequence number

In financial services, the important question is not just “does it support window functions,” but:

- Does it behave consistently with ANSI expectations?
- Are frame semantics easy for the team to reason about?
- Are timestamp and ordering edge cases handled clearly?
- Can the engine scale without changing business meaning?

For example:

- ANSI SQL-oriented platforms are usually better when you want strict relational clarity, predictable semantics, and easier audit explanation.
- Spark SQL is often stronger when the data is very large and distributed processing matters, but you must test semantics, performance, and maintainability carefully for complex windows.

So technical fit should include both:

- semantic correctness
- operational scalability

B. Migration risk

Ask: What can go wrong if we move?

This is usually more important than feature count.

Score migration risk across:

- SQL dialect differences
- window function behavior differences
- ROWS versus RANGE interpretation

- null sorting behavior
- timestamp precision and timezone handling
- reconciliation risk between old and new outputs
- downstream report changes
- model validation / audit re-approval burden
- rollback difficulty

For a financial firm, a platform that is “80% compatible” may still be unacceptable if the missing 20% affects regulatory outputs or month-end reporting.

A useful rule:

- If a window-function difference can change reported balances, rankings, alerts, or exposure calculations, treat it as high risk, not as a minor technical gap.

C. Team capability

Ask: Can the current team operate this platform safely?

This includes:

- SQL depth
- Spark engineering depth
- distributed data debugging skill
- performance tuning skill
- testing and reconciliation discipline
- DevOps/platform support maturity
- incident response readiness

A strong ANSI SQL team with limited Spark experience may deliver more reliable outcomes on a relational platform than on Spark, even if Spark looks more scalable.

Likewise, if the team already has strong Spark data engineering, then Spark SQL may become attractive for large-scale analytical pipelines.

In financial services, team skill is really a risk multiplier:

- weak skills increase implementation delay
- weak skills increase silent data-quality failures
- weak skills increase audit and reconciliation problems

3. Apply weighted scoring, but with gates first

Do not start with weighted scoring immediately. First use gate criteria.

Gate criteria

A platform should fail early if it cannot meet:

- required regulatory accuracy □ audit traceability
- security and access controls
- data lineage requirements
- reproducibility of results
- acceptable recovery / rerun behavior
- core window-function correctness on representative workloads Only platforms

that pass the gates should go into weighted scoring.

Example weighted scoring

After gating, I would weight like this for financial services:

- Technical capability: 40%
- Migration risk: 35%
- Team capability: 25% Why this order?

Because in finance:

- correctness and scalability matter first
- migration error can create regulatory or financial exposure
- team skill matters greatly, but should not rescue a technically poor choice You can

further split technical capability into:

- correctness of analytics/window behavior
 - performance at scale
 - concurrency
 - observability/debuggability
 - governance integration
-

4. Evaluate ANSI SQL vs Spark SQL specifically through window-heavy test cases

Since your context is window analytical functions, I would create a proof set of representative queries.

Examples:

- running account balances by day and transaction time
- rolling 30-day fraud indicators
- LAG of prior transaction amount per customer
- ranking counterparties by exposure within each portfolio
- moving volatility or moving average metrics
- time-bucketed trend analysis with duplicate timestamps

For each query, compare:

- output equivalence
- execution time
- explainability to auditors
- code complexity
- tuning effort
- failure/restart behavior
- ease of unit testing and reconciliation

This is where many platform decisions become obvious.

A platform may support the function syntax, but if the team must constantly work around frame semantics, ordering issues, or subtle-heavy performance problems, that is a strategic disadvantage.

5. Separate “must-run” workloads from “best-run” workloads In practice,

you may find that one platform is not ideal for everything.

For example:

- ANSI SQL platform may be best for:
 - finance-controlled reporting
 - reconciled ledger-style analytics
 - auditable

business logic o highly governed window
calculations □ Spark SQL may be best for:
o large historical backfills o data
preparation o wide event streams o massive-
scale enrichment before final governed reporting That
can lead to a hybrid decision:

- Spark for data engineering scale
- ANSI-oriented SQL warehouse or RDBMS for controlled financial analytics This is often more realistic than forcing one platform to do all jobs equally well.

6. Use a risk-adjusted recommendation, not just a feature winner At the end, I would produce a decision table like this:

Factor	ANSI SQL-style platform	Spark SQL
Window semantics clarity	High	Medium to High, depends on patterns
Audit explainability	High	Medium
Very large-scale distributed processing	Medium	High
Migration difficulty from legacy SQL	Lower	Higher
Team learning curve	Lower for SQL-heavy teams	Higher unless Spark already mature
Operational complexity	Lower	Higher
Best fit for controlled financial reporting	High	Medium
Best fit for large-scale prep / backfill	Medium	High

Then convert that into a recommendation such as:

- Choose ANSI SQL-first if the priority is auditability, predictable window behavior, and low migration risk.
 - Choose Spark SQL-first if the dominant challenge is data scale and the organization already has strong distributed-data engineering maturity.
 - Choose hybrid if reporting logic must remain tightly governed but upstream data preparation is too large for conventional SQL engines.
-

7. My recommended starting sequence

For a financial services company, I would begin in this exact order:

1. Identify the top business-critical analytical workloads.
2. Mark which ones depend heavily on window functions.
3. Define non-negotiable controls: audit, lineage, reproducibility, reconciliation.
4. Run side-by-side test cases in ANSI SQL and Spark SQL.
5. Score semantic correctness before performance.
6. Score migration risk before future flexibility.
7. Score team readiness honestly, including support and debugging skill.
8. Decide whether one platform wins, or whether a hybrid architecture is safer.

Bottom line

For financial services, the platform decision should usually be driven by this priority order:

correctness and auditability first, migration risk second, team-operability third, raw scale fourth.

Spark SQL becomes compelling when scale is the dominant problem and the team is already strong in distributed systems. ANSI SQL-style platforms usually win when the organization values predictable window-function behavior, easier reconciliation, and lower operational risk.

SECTION 2

I would usually prioritize them in this order: **business-critical use cases first, then technical risk, then migration risk, then team capability**. The reason is that the platform must first support the company's most important regulated and revenue-affecting workloads before anything else matters. After that, I would judge whether the technical design is sound, whether

the move creates unacceptable transition risk, and whether the team can realistically operate it well.

No single factor always takes precedence in every case, although **business-critical use cases and correctness** usually come first in financial services. For example, if a platform is powerful but introduces major reconciliation or audit risk, migration risk may outweigh team comfort or feature breadth. In other situations, such as a smaller non-critical workload, team capability or speed of adoption may carry more weight because the business impact of mistakes is lower.